

UD 8 - Práctica Evaluable 4

1ºDAM - PROGRAMACION

DESARROLLO DE UNA APLICACIÓN DE GESTIÓN PARA PINACOTECAS ESPAÑOLAS



PostgreSQL



SQLite



ÍNDICE

1. Introducción	3
2. Identificación del Equipo	3
3. Distribución de Roles	4
4. Diagrama Entidad-Relación (E-R)	5
5. Diagrama de Tablas de la Base de Datos	6
6. Diagrama de Clases	8
7. Justificación de Tecnologías	10
8. Estrategia de Git	13
9. Estructura del Proyecto	14

1. Introducción

El presente documento recoge la memoria detallada del proyecto que tiene como objetivo, que el alumnado desarrolle una aplicación de escritorio en Java para la gestión de pinacotecas españolas. Implementa una arquitectura robusta mediante conectividad JDBC para la persistencia de datos (PostgreSQL/SQLite), ofreciendo funcionalidades CRUD completas sobre sus entidades principales: pinacotecas, cuadros, pintores, escuelas y mecenas. El sistema ha sido desarrollado bajo un modelo colaborativo profesional, integrando el control de versiones con Git y una estructura de roles definida para asegurar la calidad y mantenibilidad del software.

2. Identificación del Equipo

Nombre del proyecto: Proyecto práctica evaluable U8.

Asignatura: Programación.

Curso académico: 1ºDAM.

Nombre completo de todos los integrantes del equipo:

- Fátima Vázquez Rivera
- Alejandro Andrade Morales
- Juan Francisco Garrido Ariza
- Diego Manuel Carrasco
- Hugo Rodríguez Viguera

3. Distribución de Roles

Para garantizar una ejecución eficiente del software y asegurar la calidad del proyecto, el equipo ha implementado una distribución estratégica de roles.

Este enfoque permite que cada integrante se especialice en un área, garantizando que todos los requisitos técnicos y documentales sean cubiertos bajo una clara asignación de responsabilidades.

A continuación, se detalla la asignación de roles y la carga de trabajo equitativa establecida:

MIEMBRO DEL EQUIPO	ROL ASIGNADO	TAREAS PRINCIPALES	PORCENTAJE DE DEDICACIÓN
Fátima	Jefe De Proyecto	Coordina el equipo, gestiona los plazos, organiza las reuniones y supervisa el cumplimiento de los objetivos. Es el responsable último de la entrega.	20%
Alejandro	Diseñador De Base De Datos	Elabora el modelo conceptual (diagrama E-R), el modelo lógico y el diseño de las tablas. Define las restricciones de integridad, claves primarias y foráneas.	20%
Juan Francisco	Responsable de Calidad y Documentación	Redacta la memoria del proyecto, revisa la calidad del código, elabora los diagramas de clases y garantiza que el repositorio y el README están correctamente mantenidos.	20%
Hugo	Desarrollador de Interfaz	Diseña e implementa la interfaz de usuario (consola o gráfica) para interactuar con el sistema. Integra las funcionalidades CRUD en la interfaz.	20%
Diego	Diseñador Backend	Implementa la lógica de negocio en Java: clases de modelo, repositorios, servicios y la capa de acceso a la base de datos mediante JDBC.	20%

4. Diagrama Entidad-Relación (E-R)

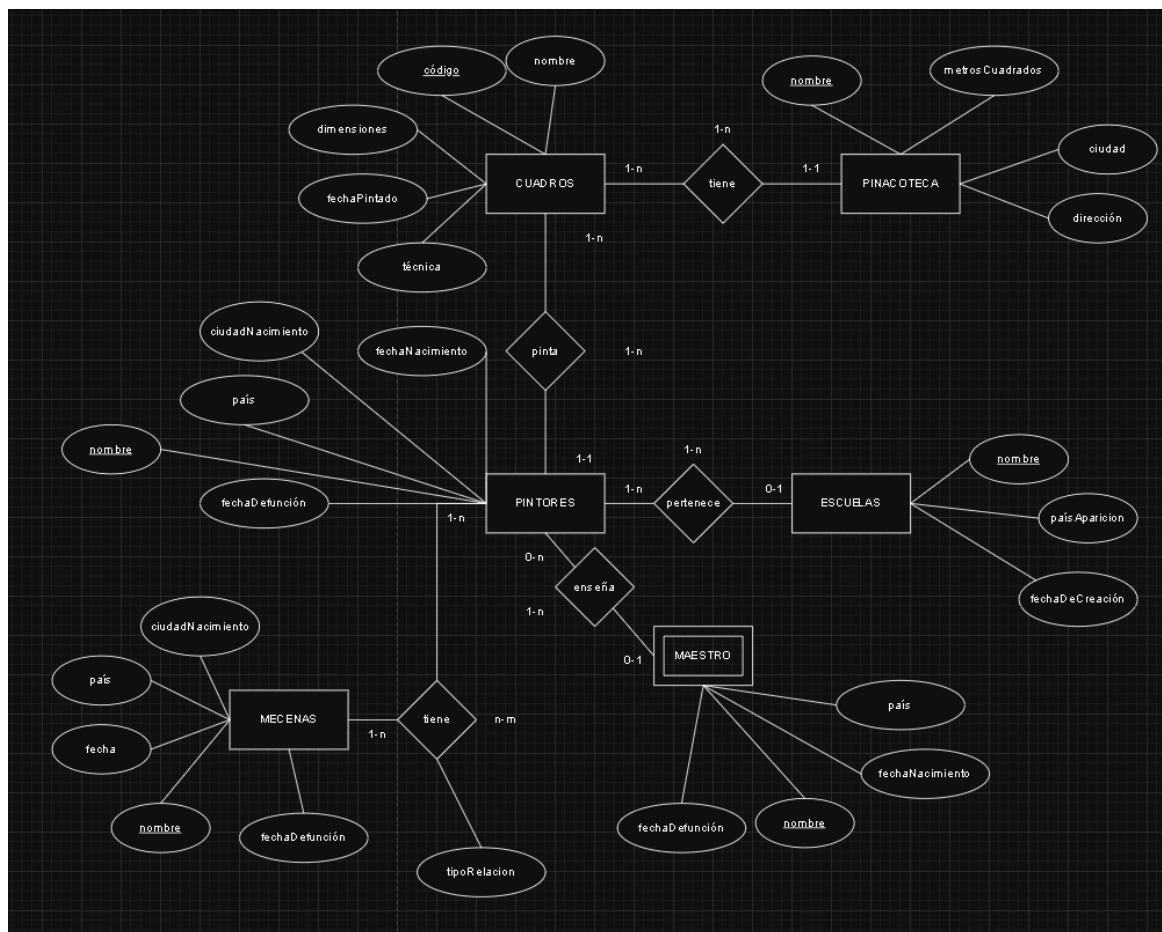
El modelo Entidad-Relación es la herramienta fundamental para diseñar cualquier base de datos. Define las reglas y las entidades principales del sistema sin preocuparse por la implementación.

Entidades Principales:

- PINTORES: El objeto central de estudio.
- CUADROS: Las obras de arte creadas.
- ESCUELAS: Los movimientos o corrientes artísticas.
- MECENAS: Los patrocinadores que financiaron el arte.
- PINACOTECAS: Los lugares de exhibición.
- MAESTRO: Una entidad especial derivada de *Pintores*.

Relaciones Lógicas:

- Pintar: Un pintor crea uno o varios cuadros.
- Pertenece: Un pintor se inscribe en una escuela artística específica.
- Tiene : Un mecenas financia a uno o varios pintores.
- Enseña: Relación jerárquica donde un pintor actúa como maestro de otro pintor.
- Ubicación: Un cuadro es albergado por una pinacoteca.



5. Diagrama de Tablas de la Base de Datos

Este diagrama es un Modelo Entidad-Relación (ER) que estructura una base de datos relacional diseñada para gestionar información sobre el mundo del arte. El objetivo es organizar los datos sobre artistas, sus obras, su formación, los lugares donde se exponen y las personas que los financiaban (mecenas).

Análisis de las Entidades

Cada tabla representa un objeto del sistema:

- pintores: Es el núcleo central que contiene información sobre los artistas.
- cuadros: Almacena los detalles de las obras de arte (nombre, dimensiones, técnica, fecha).
- escuelas: Representa movimientos artísticos, permitiendo clasificar a los pintores con atributos como fecha de creación o su país de aparición.
- mecenas: Registra a las personas que financiaban a los artistas.
- pinacotecas: Representa los museos o galerías donde se encuentran las obras.
- pintor_mecenas: Es una tabla intermedia necesaria para gestionar la relación entre pintores y mecenas.

Explicación de las Relaciones

Relaciones de Uno a Muchos (1:N)

- Escuelas → Pintores: Una escuela puede tener muchos pintores asociados, pero cada pintor pertenece a una sola escuela.
- Pintores → Cuadros: Un pintor puede haber creado muchos cuadros, pero cada cuadro está asociado a un pintor específico.
- Pinacotecas → Cuadros: Una pinacoteca puede albergar muchos cuadros, pero cada cuadro se ubica en una sola pinacoteca.

Relación de Muchos a Muchos (N:M)

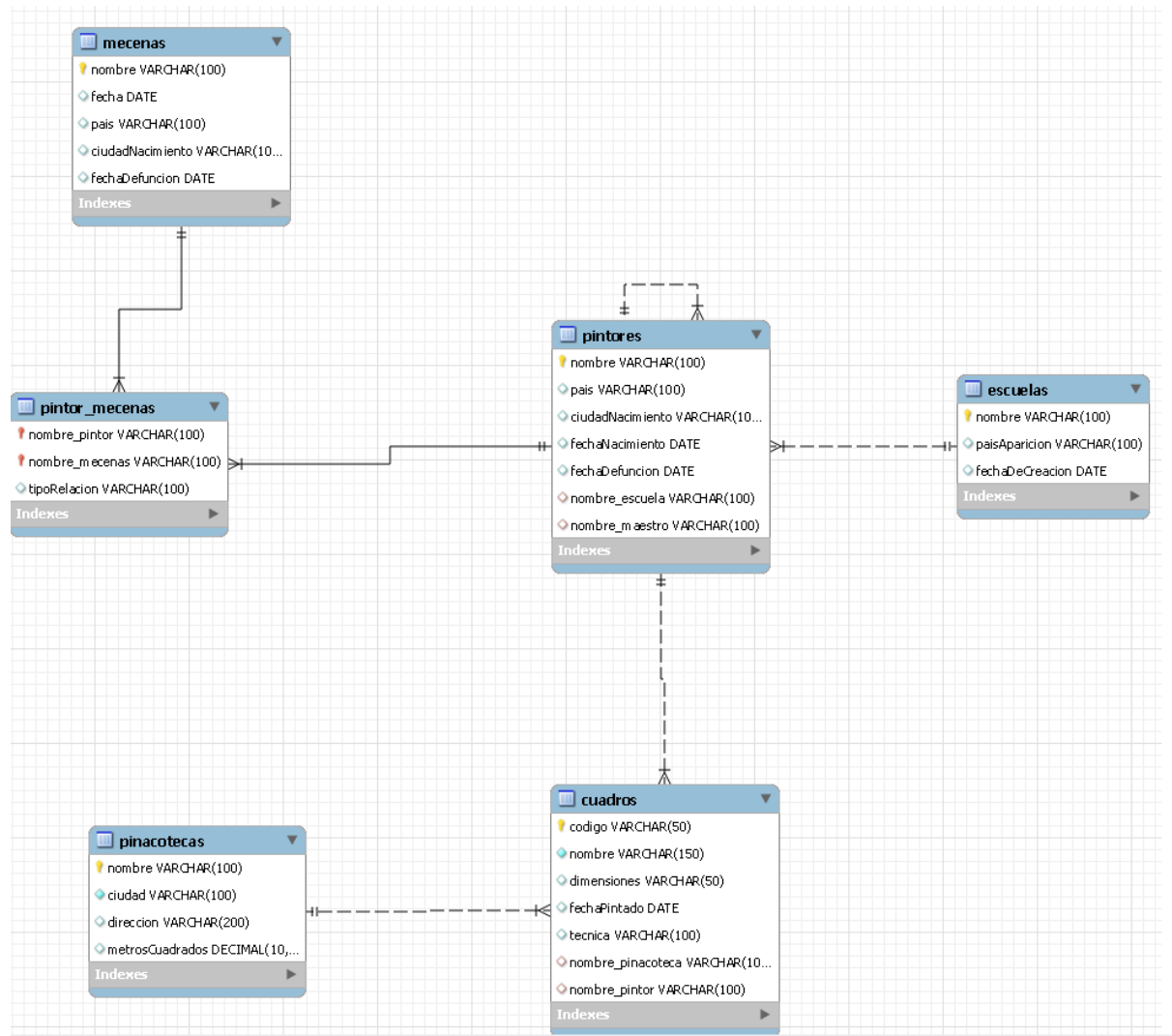
- Pintores ↔ Mecenas: Un pintor puede tener varios mecenas, y un mecenas puede financiar a varios pintores.

Como las bases de datos relacionales no pueden manejar directamente una relación "muchos a muchos", se utiliza la tabla pintor_mecenas.

- Un pintor tiene muchas entradas en pintor_mecenas.
- Un mecenas tiene muchas entradas en pintor_mecenas.

Relación Recursiva

- Pintores → Pintores: En la tabla pintores se relaciona consigo misma, referenciando un registro a otro dentro de la misma entidad (a través del campo nombre_maestro). Esto indica una relación jerárquica, es decir, un pintor puede ser maestro de otro.



6. Diagrama de Clases

El diagrama UML de clases representa la arquitectura del proyecto "Pinacoteca", detallando la estructura de los objetos, sus atributos, métodos y las relaciones de dependencia y realización que permiten el funcionamiento de la aplicación.

El sistema se organiza en cuatro capas claramente diferenciadas:

- Capa de Conexión: Clase que gestiona la sesión con SQLite, esencial para que la capa DAO interactúe con la base de datos.
- Capa de Modelo: Incluye las clases Mecena, Pintor, Cuadrado (Cuadro), Pinacoteca y Escuela. Son las representaciones lógicas de las tablas de la base de datos. Utilizan encapsulamiento (atributos privados) para proteger la información, permitiendo la gestión de los datos mediante métodos getters y setters
- Capa de Acceso a Datos:
 - Interfaces (MecenaDao, PintorDao, etc.): Definen el CRUD y el método crearTabla().
 - Implementaciones (MecenaDaoImpl, PintorDaoImpl, etc.): Contienen la lógica SQL que implementan las interfaces mencionadas previamente.
- Capa de Control: Está formada por la clase Main y los distintos Menús(MenuMecena, MenuPintor, etc.). Su función es gestionar las interacciones con el usuario y la clase Main inicializa el programa.

Además, se observan los siguientes tipos de relaciones:

- Herencia:
Las clases Impl (ej. CuadradoDaoImpl) implementan sus interfaces (ej. CuadradoDAO). Esto permite el polimorfismo y la definición correcta del comportamiento de su implementación técnica.
- Dependencia:
Los DAOs dependen del Modelo. Utilizan las clases del paquete(directorio) modelo. Los Menús dependen de los DAOs, es decir llaman a los métodos de ellos para procesar las peticiones del usuario.
- Asociación/Composición:
Se establece entre las entidades del modelo. Por ejemplo, la relación entre Pintor y Escuela o Cuadrado y Pintor, que define la jerarquía de datos.

- En el Modelo: predominan los constructores sobrecargados (para crear objetos con o sin parámetros) y los métodos de acceso (getters y setters), que permiten manipular los atributos de forma segura.
- En los DAO: son métodos encargados de la base de datos que incluyen operaciones CRUD (insertar(), eliminar(), modificar(), listar()) y el método crearTabla().
- En los Menús: son métodos de interacción como registrar(), consultar() o borrar(). Su función es dirigir la lógica del programa, mediante la entrada del usuario con un objeto Scanner y enviando los datos a la capa DAO.

7. Justificación de Tecnologías

La arquitectura tecnológica del proyecto "Pinacotecas" ha sido seleccionada con el objetivo de maximizar la robustez, la mantenibilidad del código y la eficiencia en el trabajo colaborativo.

Cada herramienta ha sido elegida tras una fase de análisis comparativo, y priorizando la estabilidad del sistema.

- Lenguaje de Programación: Java

Hemos optado por Java por ser un lenguaje de madurez tecnológica. Su máquina virtual de Java (JVM), garantiza la portabilidad del software (se escribe una vez, y se ejecuta en cualquier lugar) y su nivel de seguridad en tiempo de compilación que minimiza errores lógicos cruciales en una aplicación.

Ventajas experimentadas:

La estructura orientada a objetos encajó perfectamente con la idea de representar cada entidad (Pintor, Cuadro, Pinacoteca...) como una clase con sus datos y sus métodos.

Dificultades y soluciones:

Al principio nos costó organizar bien los paquetes y decidir qué responsabilidad tenía cada clase. Lo fuimos resolviendo a medida que avanzaba el proyecto y veíamos cómo encajaban las piezas entre sí.

- Sistema gestor de base de datos (PostgreSQL o SQLite):

Optamos por SQLite porque no requiere instalar ningún servidor: toda la base de datos se guarda en un único archivo (Pinacoteca.db) que se incluye directamente en el proyecto.

Ventajas experimentadas:

Al estar el archivo dentro del propio proyecto, cualquier miembro podía clonar el repositorio y ejecutar la aplicación sin configurar nada extra. Además, IntelliJ permite abrir y consultar ese archivo directamente, lo que nos ayudó a comprobar si los datos se estaban guardando bien.

Dificultades y soluciones:

SQLite no comprueba las claves foráneas por defecto, así que los borrados en cascada no funcionaban. Buscando encontramos que había que activarlo manualmente con `PRAGMA foreign_keys = ON` cada vez que se abre la conexión.

- **Conector JDBC / librería de persistencia utilizada**

Para que Java pudiera hablar con la base de datos SQLite usamos el driver sqlite-jdbc de Xerial, que añadimos como dependencia en el pom.xml. Organizamos todo el acceso a datos siguiendo el patrón DAO, que habíamos visto en clase: una interfaz con los métodos y una clase que los implementa, una por cada entidad.

- **Ventajas experimentadas:**

Separar la lógica de base de datos en clases DAO fue muy útil para repartir el trabajo: cada miembro se encargó de su entidad sin meterse en el código de los demás. También nos evitó tener consultas SQL dispersas por todo el código.

- **Dificultades y soluciones:**

Al principio olvidábamos cerrar la conexión después de usarla, lo que generaba errores extraños. Lo solucionamos usando try-with-resources, que cierra los recursos automáticamente al terminar.

- **Entorno de desarrollo (IDE):**

Elegimos IntelliJ IDEA porque es el IDE que usamos durante todo el curso y con el que más cómodos nos sentimos. Al ser el estándar en el módulo, todos los miembros del equipo partían del mismo entorno sin necesidad de adaptarse a nada nuevo.

- **Ventajas experimentadas:**

IntelliJ nos avisaba de los errores antes de ejecutar el programa, lo que ahorra mucho tiempo. También nos facilitó navegar entre clases, renombrar métodos en todos los sitios a la vez y ver la base de datos sin salir del IDE.

- **Dificultades y soluciones:**

En algunos ordenadores el proyecto no reconocía la versión de Java al importarlo y daba errores raros. Se solucionaba yendo a File > Project Structure y seleccionando el JDK correcto manualmente.

- **Herramientas adicionales: Maven**

Para organizar nuestro proyecto y manejar las librerías necesarias, utilizamos Apache Maven. En el desarrollo de software, lo habitual es necesitar archivos externos (librerías) para que el código funcione. Antes, esto se hacía bajando archivos .jar de Internet y metiéndolos en carpetas a mano, lo cual es muy desordenado y da muchos errores. Lo usamos para dejar de gestionar las librerías manualmente. Con Maven, solo tenemos que escribir qué librerías necesitamos en un archivo llamado 'pom.xml', y la herramienta se encarga de descargarlas y conectarlas al proyecto automáticamente. Su implementación aseguró la estabilidad del proyecto al eliminar conflictos de versiones, garantizando que todos los desarrolladores trabajarán bajo una configuración de librerías idéntica (mismas dependencias en el archivo de configuración establecidas en Git).

- **Ventajas experimentadas:**

La estructura de carpetas que impone Maven (src/main/java, etc.) nos ayudó a tener el proyecto ordenado desde el principio y a saber siempre dónde estaba cada cosa.

- **Dificultades y soluciones:**

Cuando añadimos mal una dependencia el proyecto entero dejaba de compilar y no sabíamos por dónde empezar. Lo fuimos resolviendo leyendo la documentación y buscando información.

- **Herramientas adicionales: librerías externas, frameworks de interfaz gráfica, etc.**

No utilizamos ninguna librería adicional más allá de sqlite-jdbc. Decidimos no añadir interfaz gráfica y mantener la aplicación en consola, ya que el objetivo del proyecto era practicar el acceso a base de datos y el patrón DAO, no el diseño visual.

- **Ventajas experimentadas:**

No depender de librerías adicionales mantuvo el proyecto simple y sin problemas de compatibilidad. Cualquier miembro podía ejecutarlo sin instalar nada más allá de lo declarado en el pom.xml.

- **Dificultades y soluciones:**

Precisamente por no usar ninguna librería de interfaz gráfica, toda la interacción con el usuario hubo que construirla a mano con Scanner y System.out.println. Mostrar los datos de forma clara y legible en consola requirió más trabajo del esperado, ya que no disponíamos de componentes visuales que lo facilitaran

8. Estrategia de Git

En este proyecto, el control de versiones no solo ha servido para guardar el código, sino que ha sido un control de nuestra colaboración y abarcarlo con más limpieza, control y organización. A continuación, se detalla la metodología aplicada:

Flujo de trabajo adoptado

Iniciamos el proyecto con un Commit Inicial en la rama MAIN, que contenía la estructura base de carpetas (app, conexion, dao, modelo) y los archivos pom.xml y README. A partir de ahí, el flujo evolucionó de la siguiente manera:

- Cambio de estrategia: Inicialmente, planteamos crear una rama por cada integrante. Sin embargo, detectamos que esto aislaba demasiado el trabajo. Para fomentar la colaboración y evitar que alguien se sintiese "solo" ante su parte, cambiamos a una estrategia basada en roles.
- Ramas por rol: Se crearon ramas específicas para cada capa del proyecto (ej. JefeDeProyecto, Desarrolladores, ...). Esto permitió que, si un compañero terminaba su tarea o necesitaba apoyo, otro pudiera entrar en esa misma rama para colaborar, facilitando el apoyo mutuo y ser un buen equipo.
- Integración: Cada vez que se completaba una fase del proyecto, se realizaba un Merge hacia la rama principal o hacia la rama del compañero que dependía de ese código, asegurando que todos trabajáramos con la versión más reciente.

Justificación de la estrategia

Elegimos este modelo por tres razones principales:

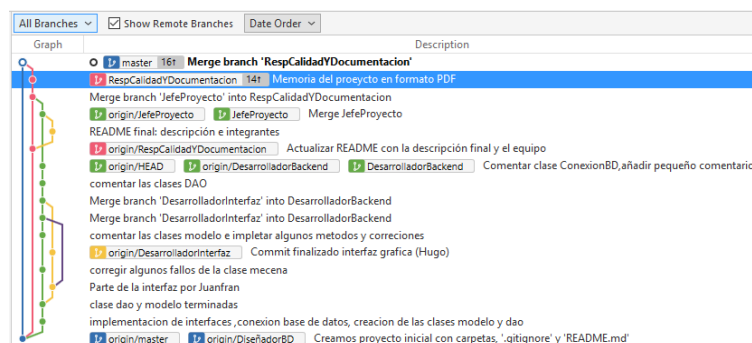
1. Visibilidad: Todos sabíamos en qué parte del sistema (rol) se estaba trabajando.
2. Solidaridad: Evitaba que un integrante se quedara bloqueado; si algún compañero terminaba, se "saltaba" a la rama de un compañero para poder ayudarlo a con su parte.
3. Orden: Mantenía la rama MAIN limpia, utilizándose solo para versiones que ya funcionaban perfectamente.

Reflexión sobre el control de versiones

A diferencia de otros proyectos, no se presentaron conflictos de fusión (conflictos con Merge) ni errores críticos.

Esto se logró gracias a la realización de commits pequeños y frecuentes y a una comunicación constante hacia la jefa de proyecto, antes de cada unión de ramas.

El paso de "ramas por persona" a "ramas por rol" fue decisivo, ya que convirtió el desarrollo en un verdadero esfuerzo de equipo donde todos ayudaban en todas las partes necesarias.



9. Estructura del Proyecto

El software se organiza siguiendo una arquitectura que separa la lógica de presentación, el acceso a datos y el modelo de objetos. A continuación se detalla el árbol de directorios completo y una explicación brevemente de cada componente.

